

Observed Recoverable Behavioral Failure Modes in LLM Workflows

A Multi-Session Cross-Platform Case Study, Retry-Probe Pilot, and Multi-Model Evaluation Protocol

Vijay Suresh

Version 1.14 · May 2026

Abstract

Standard large-language-model evaluations primarily score terminal outputs: whether a final answer is correct, safe, preferred, or useful. This paper studies a different class of failures, which we call recoverable behavioral failures (RBFs). An RBF occurs when a model appears to possess the capability, context, and tool access needed to complete a task but does not exercise that capability on the first pass, and subsequently succeeds after a brief retry, verification, or correction prompt that introduces no new substantive information. RBFs are observable in long, practical AI workflows including research synthesis, source verification, document drafting, code generation, multimodal interpretation, and artifact production. They are not captured by single-turn benchmark accuracy or aggregate human preference data, because they only become visible across turns, when a second prompt reveals that the underlying capability was already available.

The paper makes six contributions. First, it defines recoverable behavioral failure as a distinct evaluation construct, separating capability from first-pass execution behavior. Second, it develops a layered sixteen-mode taxonomy spanning retrieval, reasoning, multimodal interpretation, infrastructure, artifact production, and editorial boundary control. Third, it reports a retry-probe pilot on a fifteen-item heterogeneous retrieval task in which first-pass resolution was 12/15 and after-retry resolution was 15/15, yielding a 20 percentage-point recoverability gap. Wilson 95% confidence intervals are reported for the small-sample rates, and an exact small-sample Fisher test associates first-pass failure with one structural item class (two-sided $p = 0.0022$, exact). Fourth, it extends the taxonomy with abstracted cross-platform workflow observations covering artifact readiness, source verification, audience calibration, and editorial boundary modes. Fifth, it specifies a controlled cross-vendor evaluation protocol grounded in a published task pack, with task families, deterministic and human-judged checks, sample-size guidance, an annotation rubric, and a reproducibility checklist. Sixth, it reports a small intra-vendor pilot on the ten-task experiment pack across three Anthropic models, with per-model and per-category pass rates and Wilson 95% confidence intervals. The pilots are exploratory; the cross-vendor protocol is offered as planned replication rather than as a completed cross-vendor study.

Keywords: large language models; evaluation; behavioral failures; retry behavior; agent reliability; tool use; artifact generation; self-verification; workflow readiness; case-study methodology; reproducible benchmark.

Keywords: large language models; evaluation; behavioral failures; retry behavior; agent reliability; tool use; artifact generation; self-verification; workflow readiness; case-study methodology; reproducible benchmark.

Contents

1 Introduction	3
2 Related Work	4
3 Conceptual Framework	6
4 Data and Methodology	7
5 Taxonomy of Recoverable Behavioral Failures	8
6 Cross-Workflow Observations	20
7 Pilot Experiment: Retry-Probe Quantification of PTS	21
8 Intra-Vendor Pilot Results: Sonnet, Opus, and Haiku	22
9 Observational Notes Beyond the Pilot	24
10 Proposed Cross-Vendor Evaluation Protocol	25
11 Companion Repository and Reproducibility Package	26
12 Mitigation Implications	27
13 Limitations and Threats to Validity	28
14 Future Work	29
15 Conclusion	30
References	31
Appendices	33

1 Introduction

Public discussion of language-model failures has converged on two broad categories. A capability failure occurs when the model lacks the knowledge, reasoning skill, perceptual ability, or tool fluency required to solve the task. An alignment or safety failure occurs when the model produces output that is harmful, deceptive, or otherwise off-policy. Existing evaluation infrastructure is built around these two categories: benchmark accuracy (Liang et al., 2022; Srivastava et al., 2023; Hendrycks et al., 2021) indexes the first; preference-based reinforcement learning (Christiano et al., 2017; Bai et al., 2022) and red-teaming index the second.

Many failures observed in practical AI workflows fit neither category cleanly. A user issues a request; the model declares part of the task unresolved, incomplete, or impossible; a short follow-up prompt elicits a substantively correct response without introducing new documents, tools, or facts. The capability was demonstrably available on the first turn. What changed between attempts was a search strategy, a verification step, a recognition of structural input, or an audit of an artifact for placeholders, formatting, or boundary leakage. The first attempt did not fail because the model could not do it. It failed because the model did not.

We refer to these events as recoverable behavioral failures. They are behavioral not in any anthropomorphic sense but because the failure is located in execution policy: effort allocation, persistence, verification, escalation, search discipline, multimodal reading, artifact finalization, and boundary management between conversation and deliverable. They are recoverable because a local correction reveals the model was capable of producing the correct output without additional information.

The distinction matters for modern AI workflows. Frontier models are now used for research synthesis, software debugging, document drafting, slide and spreadsheet generation, literature triage, citation checking, and tool-assisted operations. These are not single-turn tasks with a single final answer. They are multi-step, supervised processes in which the practical cost of a model is not only the probability of a wrong terminal answer; it is the amount of human oversight needed to detect premature stopping, recover incomplete artifacts, correct unsupported claims, and prevent process commentary from contaminating the final deliverable.

This paper is intentionally conservative. It does not estimate population-level failure rates for any vendor, model family, or product. It develops the construct, codes a naturalistic multi-session corpus, measures one mode quantitatively in a small retry-probe pilot, reports a small intra-vendor model pilot on a ten-task

experiment pack, and specifies a cross-vendor evaluation protocol that future work can run to replicate and extend the findings.

1.1 Contributions

- **Construct.** A definition of recoverable behavioral failure that separates capability from first-pass execution behavior, with operational measurement instruments: first-pass success, retry success, and recoverability gap.
- **Taxonomy.** A layered sixteen-mode taxonomy organized by failure locus.
- **Pilot.** A retry-probe pilot with first-pass rate $12/15 = 80\%$ (Wilson 95% CI: [54.8%, 93.0%]), retry rate $15/15 = 100\%$, and recoverability gap +20 percentage points (discordant-proportion CI: [7.0%, 45.2%]). Exact Fisher test on item-structural class association: two-sided $p = 0.0022$.
- **Cross-workflow extensions.** Four additional modes from professional artifact-production and public-communication workflows.
- **Cross-vendor protocol.** A controlled benchmark specification grounded in a ten-task pilot pack covering five task categories.
- **Intra-vendor pilot results.** A pilot run of the experiment pack across three Anthropic models producing per-model and per-category pass rates and Wilson 95% confidence intervals. The result is reported as a pilot with rubric-vocabulary caveats, not as a population estimate or model ranking.

2 Related Work

2.1 LLM benchmarks and evaluation

HELM (Liang et al., 2022), BIG-Bench (Srivastava et al., 2023), and MMLU (Hendrycks et al., 2021) established broad coverage of LLM capabilities across reasoning, knowledge, and language tasks. Their primary measurement target is correctness, calibration, or preference of a terminal output under a fixed prompt. They do not separate the gap between what a model produces on the first attempt and what it can produce after explicit retry, verification, or an alternative search strategy. This paper complements terminal-output evaluation by treating first-pass and post-retry performance as separately measurable quantities and by adding artifact-level readiness checks alongside text-level correctness.

2.2 RLHF and preference learning

Reinforcement learning from human feedback (Christiano et al., 2017; Bai et al., 2022) and direct preference optimization (Rafailov et al., 2023) aggregate pairwise human preferences over candidate responses. Preference labels are informative about which final answer is preferred, but they do not by themselves encode the reason for the preference. If the dispreferred answer would have improved under a follow-up retry without new information, that recoverability is not captured in the preference signal.

2.3 Tool-use and agentic benchmarks

SWE-bench (Jimenez et al., 2024), WebArena (Zhou et al., 2024), Toolformer (Schick et al., 2023), ReAct (Yao et al., 2023), WebGPT (Nakano et al., 2021), Gorilla (Patil et al., 2023), ToolLLM (Qin et al., 2023), AgentBench (Liu et al., 2023), and GAIA (Mialon et al., 2023) evaluate complex tool-use and agentic behavior. These benchmarks measure end-to-end task completion, but they do not always isolate intermediate execution decisions: when to retry a thin search, when to escalate query construction, when to verify a numerical claim, when to ask for structural confirmation of a multimodal input, and when to declare an artifact complete.

2.4 Truthfulness, sycophancy, hallucination, and deception

TruthfulQA (Lin et al., 2022) targets the assertion of false-but-popular claims. Sycophancy work (Sharma et al., 2023) documents the tendency to align responses with stated user preferences even when those preferences diverge from the correct answer. Surveys of hallucination and deception (Ji et al., 2023; Park et al., 2024) capture confidently wrong behavior. The present taxonomy intersects this literature but extends to a complementary surface pattern: the model may decline, truncate, misread, or produce an incomplete artifact rather than confidently assert a false proposition.

2.5 Time-sensitive QA and knowledge staleness

Time-sensitive question answering (Chen et al., 2021; Liska et al., 2022) documents that static model knowledge degrades as the world changes. This paper extends the concern from factual decay to procedural decay. When a model confidently describes a third-party tool interface that has been restructured, the resulting failure is not a factual hallucination in the traditional sense; it is a stale procedural assumption that wastes user effort.

2.6 Long-horizon agent reliability and human-AI workflow reliability

A growing body of work documents the gap between single-turn capability and multi-turn reliability. Agent benchmarks report success rates that decline with task length, number of tool calls, and number of subgoals, even when single-step accuracy on the same operations is high. Studies of human-AI collaboration on knowledge-intensive tasks document supervisory load when assistants fail in recoverable but uncaught ways: users absorb the cost of detecting premature stopping, re-issuing corrective prompts, and auditing artifacts whose surface form looks acceptable. The RBF construct offers a unified measurement language for this observation.

2.7 Case-study methodology

The empirical structure follows theory-building case-study logic (Yin, 2018). Naturalistic cases are appropriate when the phenomenon is multi-turn, workflow-dependent, and not yet reducible to isolated benchmark questions. The role of the case study is not to replace controlled experimentation; it is to discover and characterize constructs that can then be measured through controlled replication.

3 Conceptual Framework

We define a recoverable behavioral failure relative to a verifiable task. Let an item i in task set T have a verifiable target outcome y_i . Let $A_1(i)$ denote the model’s first-pass attempt under the original prompt, and let $A_r(i)$ denote the model’s attempt after a standardized retry prompt that supplies no new substantive information: no new documents, no new tools, no new facts, no new task specification, and no relaxed evaluation standard. Let $S_1(i) = 1$ when $A_1(i)$ satisfies the verification criterion, and analogously $S_r(i)$ for the retry attempt.

3.1 Core quantities

First-pass success rate: $p_1 = (1/N) \sum_{i=1}^N S_1(i)$.

Retry success rate: $p_r = (1/N) \sum_{i=1}^N S_r(i)$.

Recoverability gap: $G = p_r - p_1$.

RBF indicator for item i : $I[S_1(i) = 0 \text{ and } S_r(i) = 1]$ under the standardized retry. Aggregating the indicator across items yields the empirical RBF rate.

3.2 Construct boundaries

The construct is intentionally conservative. The following are not counted as RBFs:

- Retries in which the user supplies a missing fact, document, or answer.
- Retries in which the task specification or evaluation standard is relaxed.
- Retries in which new tool affordances become available between attempts.
- Cases where the first-pass output is correct under a reasonable reading and the user demands an unnecessary rephrase.

These exclusions protect the construct from inflation. Without them, every multi-turn dialogue would appear to contain behavioral failures; with them, the construct isolates the narrower phenomenon where an already available capability is not exercised on the first pass.

3.3 Additional measurable constructs

- Batch-scaling degradation: the change in per-item first-pass success as batch size N varies.
- Artifact-readiness failure: a deliverable exists but does not meet user-facing readiness criteria.
- Boundary-contamination failure: process commentary appears inside a deliverable that should be publication-ready.
- Verification compliance: the proportion of items for which a stated verification step is performed when implied by the task.

4 Data and Methodology

4.1 Evidence base

The evidence combines a fourteen-session naturalistic case study of multi-turn workflows (document generation, source verification, multi-step research, quantitative reasoning, structured visual interpretation, publication preparation) with a cross-platform abstraction over additional professional workflows. The paper reports only abstracted workflow categories and failure mechanisms, with no individual identifying details.

4.2 Evidence tiers

Observations are assigned to evidence tiers to prevent overstating individual claims: Tier 1 measured pilot item outcomes; Tier 2 transcript-verifiable multi-turn observations; Tier 3 abstracted cross-platform workflow summaries; Tier 4 hypothesized mechanisms requiring controlled replication.

4.3 Coding procedure

Each candidate event is coded with task context, original output, user retry, post-correction outcome, whether new substantive information was introduced, failure mode, trigger, recovery type, and user-visible cost. If recovery requires new information from the user, the event is excluded from RBFs.

4.4 Model-attribution caveats

The Claude.ai conversation export schema does not record per-message model identity. Attribution in the case study is reconstructed from user recall and contextual cues. The paper therefore avoids strong model-comparison claims based on the case study; the experimental protocol is the appropriate vehicle for controlled model comparison.

4.5 Responsible data handling

Raw transcripts are not released. The accompanying benchmark package contains items synthesized from neutral domains and does not incorporate user-specific identifiers. Future public releases should follow a two-layer model: a public benchmark from neutral sources, and a private audit appendix for trusted reviewers.

5 Taxonomy of Recoverable Behavioral Failures

The taxonomy is layered by failure locus. Locus separation matters because not every user-visible failure should be attributed to the model core. Some failures arise in retrieval policy, some in batch-effort allocation, some in artifact generation, some in client transport, some in product memory, and some in the evaluation pipeline itself. A useful framework should locate the failure precisely enough to guide the right mitigation.

Code	Name	Primary locus	User-visible cost
PTS	Premature Termination of Search	Model / retrieval policy	False unresolved answers
BDTF	Breadth-Depth Tradeoff Failure	Model policy under batch load	Shallow per-item analysis
SMI	Search Methodology Inconsistency	Retrieval strategy	Inconsistent escalation
MRT	Mid-Response Truncation	Infrastructure	Lost progress
MSI	Mobile Streaming Interruption	Client / transport	Dropped responses
FPG	Feedback Persistence Gap	Product / training pipeline	Corrections not preserved
RFI	Retry-Failure Invisibility	Evaluation pipeline	Cannot-solve vs. did-not-try
STK	Stale Third-Party Tool/UI Knowledge	Model knowledge / tool guidance	Confident wrong UI paths
QI	Quantitative Inconsistency	Reasoning / self-audit	Contradictory numbers
PPG	Phantom Placeholder Persistence	Artifact / code generation	Placeholders as commands
UCT	Usage-Cap Termination	System / rate limit	Degraded resume state
VMC	Visual Misreading Cascade	Multimodal interpretation	Cascading misreadings
AFF	Artifact Finalization Failure	Artifact generation	File exists but unusable
SVD	Source Verification Drift	Citation / retrieval practice	Weakly supported claims
EBC	Editorial Boundary Contamination	Artifact boundary management	Process notes leak in
ACM	Audience / Context Miscalibration	Communication / register	Off-register content

Table 1. Layered taxonomy of recoverable behavioral failure modes.

5.1 A note on mode overlap and granularity

Sixteen modes is a deliberate starting partition. Several pairs are adjacent and could in principle be collapsed: PTS and SMI are retrieval-policy failures; MRT, UCT, and MSI are interruption-style failures at different loci; AFF and EBC are artifact-side failures. We keep them separate because the mitigation implied by each mode is different, the locus attribution differs, and empirically driven collapse should follow from data once cross-model and cross-user evidence accumulates.

5.2 Premature Termination of Search (PTS)

The model declares an information item unresolved after a small number of retrieval attempts, even though a readily available alternative retrieval strategy would resolve it. In the pilot retrieval task, three items whose URLs contained opaque numeric identifiers were marked unresolved after generic keyword queries returned thin results. After a standardized retry prompt the model used site-targeted query construction and resolved all three. PTS matters because it masquerades as missing knowledge: the user receives an unknown response and treats it as a capability boundary, when the system has not exhausted available retrieval policy. PTS differs from SMI, which is inconsistency across similar subtasks, and from RFI, which is the evaluation blind spot that hides the recoverability of the failure. Detection uses retry probes constructed so that a naive first query is thin but an alternative strategy succeeds. Mitigation is an explicit retrieval escalation policy: query rewriting, site restriction, exact-string identifier search, and source triangulation before declaring failure.

5.3 Breadth-Depth Tradeoff Failure (BDTF)

When presented with a large batch, the model implicitly optimizes for covering all items rather than verifying each item deeply. The result is a superficially complete response with weak per-item grounding. Multi-item evaluation tasks elicit short, uniform dispositions even when individual items differ materially in difficulty; the user infers uniform depth from uniform output length, which the model has not earned. BDTF differs from MRT, which is truncation within a single response, and from PTS, which is premature stopping on a single retrieval. Detection uses batch-scaling curves: administer the same items at $N = 1, 5, 15, 30$ and measure per-item correctness against N . Mitigation is batch-aware effort budgeting, per-item uncertainty, and explicit flags distinguishing verified, partially verified, and unverified outputs.

5.4 Search Methodology Inconsistency (SMI)

The model uses stronger retrieval tactics for some items and weaker tactics for similar items within the same session, with no explicit reason for the difference. Query construction varies between generic keyword search, site-targeted search, and identifier-anchored search without a stated escalation rule. This shifts escalation burden to the user and makes retrieval discipline unpredictable. SMI differs from PTS because it is a consistency mode across subtasks rather than a failure within a single subtask. Detection uses per-subtask query logging and the conditional probability of switching strategy given a thin first result. Mitigation is a documented escalation protocol applied uniformly across retrieval subtasks.

5.5 Mid-Response Truncation (MRT)

The model begins generating a response, makes partial progress, and the stream terminates before completion, often after partial tool use or partial drafting. Long multi-part responses sometimes terminate with a generic recovery message and partial work is not always preserved. MRT is costly because users must reconstruct what the model had already done and re-prompt for the remainder. It differs from UCT, which is rate-limit-driven, and MSI, which is client-side. Detection uses near-budget telemetry: a spike of terminations near token, tool-call, or wall-clock limits indicates MRT. Mitigation is a structured progress ledger with resume-state checkpoints.

5.6 Mobile Streaming Interruption (MSI)

Mobile or client-side interruptions can drop an in-progress streaming response with no recovery of the partial output when the user returns. Screen-lock, backgrounding, and connection drops are common triggers. MSI is client/transport-layer; MRT is infrastructure/model-side; UCT is system rate-limit. Detection uses client-side telemetry on stream-drop events correlated with foreground/background lifecycle. Mitigation is persistent server-side response buffering with client reattachment on resume.

5.7 Feedback Persistence Gap (FPG)

Within-session corrections, scope clarifications, and explicit user preferences do not persist beyond the session unless an explicit memory surface or product mechanism captures them. FPG is not always a defect because user control and privacy require limits on persistence, but repeated corrections impose longitudinal cost. Detection is longitudinal: time-to-recurrence of a corrected error and proportion of stable preferences that survive across sessions. Mitigation is explicit per-user memory with user-visible controls and correction events treated as weighted product signal.

5.8 Retry-Failure Invisibility (RFI)

Standard offline evaluation records a failed turn but not the counterfactual that the same model would have succeeded under a standardized retry prompt. Capability failures and recoverable behavioral failures are statistically conflated. RFI is the evaluation-pipeline mode in the taxonomy: measuring the other modes requires first making recoverability visible. Detection uses counterfactual replay of failed turns with explicit retry framing. Mitigation is to report first-pass and retry-after-prompt success as separate metrics and track recoverability gap alongside accuracy.

5.9 Stale Third-Party Tool/UI Knowledge (STK)

The model provides confident instructions for third-party tools or interfaces whose live UI has changed since training cutoff. Instructions are plausible but cannot be followed because labels, menus, or setup flows differ. STK is procedural staleness rather than factual staleness. Detection uses confidence-decay scoring for third-party-tool navigation outputs, weighted by time since training cutoff and UI-change cadence. Mitigation is default verification framing: present live-tool instructions as estimates, suggest checking current documentation, and invite correction from the user's live UI.

5.10 Quantitative Inconsistency (QI)

Within a single response, the model emits two or more numerical claims that contradict each other under its own assumptions. The contradiction is internally checkable but not self-audited. QI is costly because the answer looks analytical and can propagate into downstream artifacts. It differs from SVD because QI requires no external source check; it requires arithmetic verification. Detection uses programmatic extraction of numerical claims and relationships. Mitigation is a numeric self-audit pass before returning responses with multiple quantitative claims.

5.11 Phantom Placeholder Persistence (PPG)

The model issues a command, file path, citation field, configuration value, or code snippet containing an unresolved template placeholder even when the surrounding context implies a concrete value is needed. PPG silently shifts work to the user, who discovers the failure on execution. Detection uses regex scans for placeholder patterns in code and command outputs. Mitigation is to substitute from session context when unambiguous or explicitly mark the placeholder and prompt the user.

5.12 Usage-Cap Termination (UCT)

A rate limit or usage cap terminates the conversation mid-task; recovery degrades to a malformed or non-substantive continuation rather than graceful state preservation. UCT is system-level, but its effect is workflow-integrity loss. Detection uses telemetry on cap events and the success rate of subsequent continuation turns. Mitigation is graceful resume design: preserve a structured progress ledger, summarize completed work, and provide a clean resume entry point when the cap clears.

5.13 Visual Misreading Cascade (VMC)

Given a structured visual input, the model produces a confident interpretation; when the user corrects a structural element, the model produces a new interpretation that is itself incorrect in a different way. VMC matters because downstream analysis depends on the misread structure. It differs from PTS because the model does not decline; from STK because the input is user-supplied; and from QI because interpretations may be internally consistent but structurally wrong. Detection and mitigation use a visual-input verification loop: summarize structure first, then ask for confirmation before downstream analysis.

5.14 Artifact Finalization Failure (AFF)

In artifact-producing workflows, the model generates a deliverable that nominally exists but does not satisfy user-facing readiness criteria. Text may be good while the file is visually or structurally flawed. AFF differs from PPG, which concerns placeholders, and EBC, which concerns process leakage. Detection uses automated artifact checks: file opens, expected page count, heading render, no placeholders, links resolve, citations support claims, and format-specific lints pass. Mitigation is a pre-delivery artifact-readiness audit.

5.15 Source Verification Drift (SVD)

A workflow begins with source verification but drifts into plausible synthesis without sufficient source grounding. Citations may support adjacent points but not the exact claim, or may be stale for time-sensitive assertions. SVD differs from hallucination because the claim and source may both be real while their alignment weakens. Detection uses claim-source entailment checks and link-freshness audits. Mitigation is a verification pass that retains citations only when the cited span supports the exact claim.

5.16 Editorial Boundary Contamination (EBC)

Process commentary, advisory metadata, scoring or readiness language, venue strategy, or other chat-level material appears inside a publication-ready deliverable. EBC is recoverable but costly because private process leaks into public output. Detection is a pre-delivery boundary audit for advisory language and chat-only content. Mitigation is an explicit dual-channel protocol: conversation receives advisory content; artifacts receive only content intended for the final deliverable.

5.17 Audience / Context Miscalibration (ACM)

The model produces technically correct content that is poorly calibrated for the intended audience, register, or distribution channel. ACM is the bridge between substantive content quality and communicative fitness. It differs from AFF, which is format-side, and EBC, which is process-leakage-side. Detection uses audience-and-channel checks, including forbidden terms incompatible with the stated audience. Mitigation is explicit audience framing and a pre-delivery audit for off-register content.

6 Cross-Workflow Observations

6.1 Document-generation and artifact-production workflows

When the deliverable is a document, slide deck, spreadsheet, or code file, terminal text quality is necessary but not sufficient. The artifact must open cleanly, render correctly, and meet readiness criteria. AFF, PPG, and EBC concentrate in these workflows; their incidence becomes visible only when the artifact is evaluated outside the chat. A reasonable evaluation discipline treats the artifact, not the response describing it, as the primary unit of measurement.

6.2 Source-verification and multi-step research workflows

Citation-heavy and verification-heavy workflows surface SVD, PTS, and SMI together. The model may begin with disciplined source-grounding, drift into plausible synthesis as the task lengthens, and selectively escalate retrieval on some claims but not others. Claim-level verification checks are the appropriate measurement; citation presence alone is insufficient.

6.3 Public-communication and audience-targeted workflows

When an artifact is intended for a public or restricted audience, ACM and EBC become salient. Technically correct content can leak private context, off-register tone, or advisory metadata. Mitigation is policy-level rather than model-capability-level.

6.4 Quantitative reasoning workflows

QI surfaces most often when responses contain multiple quantitative claims under shared assumptions. The detection method is mechanical and underused; arithmetic verification can be implemented as a cheap post-pass or model-side self-audit.

7 Pilot Experiment: Retry-Probe Quantification of PTS

7.1 Setup

A fifteen-item heterogeneous retrieval task. Items belong to three structural classes: direct URLs with human-readable paths ($n = 9$), URLs containing opaque numeric identifiers ($n = 3$), and title-only references requiring search ($n = 3$).

7.2 Procedure

For each item, first-pass success and retry-after-prompt success were recorded from the verbatim session transcript. The retry prompt, “try a different approach,” supplied no new substantive information. Conservative coding: an item counts as an RBF only when the second-pass success demonstrates previously available capability.

7.3 Results

Metric	Value	95% CI
Total items (N)	15	-
First-pass resolved (p_1)	12/15 = 80.0%	Wilson: [54.8%, 93.0%]
Resolved after retry (p_r)	15/15 = 100.0%	Wilson: [79.6%, 100.0%]
Recoverable first-pass failures	3/15 = 20.0%	Wilson: [7.0%, 45.2%]
Recoverability gap ($G = p_r - p_1$)	+20.0 pp	Discordant CI: [7.0%, 45.2%]

Table 2. Retry-probe pilot results with Wilson 95% confidence intervals.

Item structural class	n	First-pass resolved	Rate	RBFs
Direct URL (human-readable path)	9	9	100%	0
URL with opaque numeric identifier	3	0	0%	3
Title-only (search-required)	3	3	100%	0

Table 3. Pilot first-pass performance by item structural class.

7.4 Exact small-sample statistical analysis

A two-sided Fisher exact test on the 2 x 2 contingency table yields $p = 0.0022$. The one-sided test for the hypothesis that opaque-identifier items have lower first-pass success also yields $p = 0.0022$. The result is statistically meaningful for this item set but should not be read as a population-level rate.

	First-pass success	First-pass failure	Row total
Opaque-identifier	0	3	3
Non-opaque	12	0	12
Column total	12	3	15

Table 4. Contingency table for first-pass success by item structural class.

7.5 What the pilot supports and does not support

The pilot supports the existence of recoverable behavioral failures in at least one measured retrieval workflow and supports treating first-pass and retry-after-prompt success as separately measurable quantities. It suggests, exploratorily, that opaque numeric identifiers may be a high-incidence trigger for PTS under batch load. It does not establish a population-level PTS rate for any model or vendor; it does not establish a universal opaque-identifier-to-failure relationship; and it does not establish a model ranking, because workload and model identity are not experimentally balanced.

8 Intra-Vendor Pilot Results: Sonnet, Opus, and Haiku on the Experiment Pack

To complement the single-batch retry-probe pilot in Section 7, we ran the ten-task experiment pack against three Anthropic models in parallel: claude-sonnet-4-6, claude-opus-4-7, and claude-haiku-4-5-20251001. The run is an intra-vendor pilot: it holds vendor constant and removes one common cross-vendor confound. Cross-vendor extension to OpenAI and Google model families is deferred to future work because credentials for those vendors were unavailable at pilot time.

8.1 Setup

Each model received the same ten prompts under standardized settings (max_tokens 800, default decoding; temperature omitted for compatibility with the newest Opus model, which deprecates the parameter). Each task was scored by deterministic checks embedded in the task file: forbidden-term lists, required-term lists, or numeric-expectation regexes for two arithmetic items. The pack contains two items per category in five categories: QI, PPG, EBC, ACM, and RETRY_PROXY. All 30 calls (10 tasks x 3 models) succeeded at the transport level.

8.2 Results

Model	Category	n	n pass	Pass rate	Wilson low	Wilson high
Sonnet 4.6	ACM	2	2	1.00	0.34	1.00
Sonnet 4.6	EBC	2	2	1.00	0.34	1.00
Sonnet 4.6	PPG	2	2	1.00	0.34	1.00
Sonnet 4.6	QI	2	2	1.00	0.34	1.00
Sonnet 4.6	RETRY_PROXY	2	0	0.00	0.00	0.66
Opus 4.7	ACM	2	2	1.00	0.34	1.00
Opus 4.7	EBC	2	2	1.00	0.34	1.00
Opus 4.7	PPG	2	2	1.00	0.34	1.00
Opus 4.7	QI	2	2	1.00	0.34	1.00
Opus 4.7	RETRY_PROXY	2	0	0.00	0.00	0.66
Haiku 4.5	ACM	2	2	1.00	0.34	1.00
Haiku 4.5	EBC	2	2	1.00	0.34	1.00
Haiku 4.5	PPG	2	2	1.00	0.34	1.00
Haiku 4.5	QI	2	2	1.00	0.34	1.00
Haiku 4.5	RETRY_PROXY	2	0	0.00	0.00	0.66

Table 5. Per-(model, category) pass rates with Wilson 95% confidence intervals. Two items per cell.

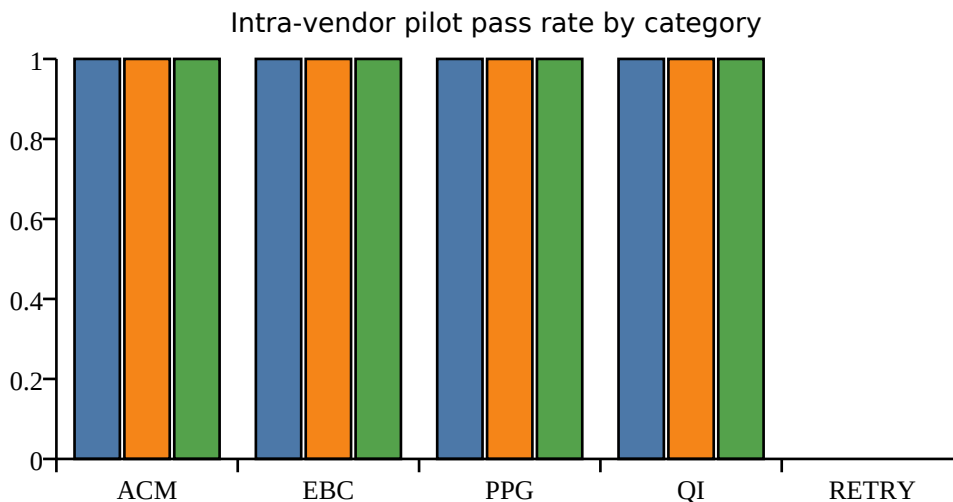


Figure 1. Multi-model pilot pass rate by category (intra-vendor). Two items per cell; confidence intervals are wide as expected.

8.3 Findings

Uniform success on artifact and audience checks. All three Anthropic models passed both items of QI, PPG, EBC, and ACM. This is an existence result for the metrics: the deterministic checks fire cleanly on responses that the rubric considers acceptable, and on these particular tasks all three models produced responses that satisfied the rubric.

Uniform failure on the RETRY_PROXY rubric. All three models failed both RETRY_PROXY items. Inspection of outputs and deterministic criteria shows that the failures are largely a rubric-vocabulary artifact: the check required specific tokens such as site, exact, identifier, and alternative to appear in the response, and the models produced reasonable retrieval-escalation strategies using different vocabulary. The failure is therefore informative about the rubric, not strictly about the models.

Implication for RFI. The RETRY_PROXY result is itself a clean instance of why retry-failure invisibility is non-trivial to measure. A strict deterministic rubric can systematically miss valid behavior; an LLM-as-judge or semantic-similarity check would catch some of the same responses as passes. This supports pairing automated scoring with double-coded human annotation, especially for retrieval-strategy and persistence-style probes where the space of acceptable answers is open.

8.4 Caveats

The sample is small: two items per cell, ten items per model. Wilson confidence intervals are wide by design. The result is intra-vendor only and cannot speak to cross-vendor differences. The deterministic-check rubric is itself a measurement artifact, as the `RETRY_PROXY` failure illustrates. We report the table to demonstrate that the protocol is implementable end-to-end across multiple frontier models and to illustrate the kind of measurement the cross-vendor protocol would scale up.

9 Observational Notes Beyond the Pilot

Beyond the measured PTS pilot, the case-study corpus contains transcript-verifiable instances of STK, QI, PPG, VMC, UCT, and BDTF. These are reported at the level of pattern, trigger, and recovery so they can be probed in controlled replication. No additional rates are claimed for these modes; they are Tier 2 observations supporting taxonomy and protocol design rather than quantitative estimates.

The purpose of this section is to preserve the observational richness that motivated the taxonomy while keeping the evidentiary status explicit. A naturalistic workflow can reveal failure structure before it can support population-level rate estimation. The contribution here is therefore not that a particular model has a known frequency of each failure mode, but that the modes are sufficiently concrete to become benchmark targets.

9.1 Stale tool and UI guidance

The STK observations show that procedural knowledge can go stale in ways that are costly even when the model is otherwise helpful. The model does not simply hallucinate a fact; it gives a plausible live-interface path that the user attempts to follow. The failure becomes visible only after the user reports that the labels, menus, or flow do not exist in the current interface. A controlled task can reproduce this condition by asking for guidance on high-churn tool surfaces and scoring whether the answer contains explicit staleness framing or current-documentation verification.

9.2 Quantitative and artifact-level failures

The QI and PPG observations show that many recoverable failures are not knowledge failures at all. They are audit failures. A response containing multiple numbers can be internally inconsistent even when each individual number sounds plausible. A command can be syntactically close to correct while retaining a placeholder that makes it unusable. In both cases, cheap deterministic checks can catch the issue before delivery, which makes these modes unusually suitable for automated evaluation and product mitigation.

9.3 Visual and interruption failures

The VMC and interruption-related observations show why workflow evaluation cannot be reduced to answer correctness. A structured visual input can be misread repeatedly across turns because the model never performs a stable extraction step before interpretation. A long task can be interrupted in ways that lose progress even when the underlying model could have continued. These modes motivate evaluation criteria around state preservation, structural confirmation, and resume quality, not merely final answer text.

9.4 Batch-load and supervision cost

BDTF is especially important because it captures a common user experience: the model appears to scale across a large batch, but per-item depth silently degrades. The user receives a complete-looking list, yet the verification burden has shifted back to them. A batch-scaling curve makes this cost visible by reporting correctness as a function of N rather than hiding it behind aggregate success on a single batch size.

10 Proposed Cross-Vendor Evaluation Protocol

This section specifies a controlled benchmark grounded in the ten-task pilot pack and designed to extend the intra-vendor pilot in Section 8 to a larger cross-vendor study. Section 8 demonstrates that the experiment pack can run end-to-end across multiple Anthropic models. The full cross-vendor protocol across Claude, GPT, Gemini, and additional frontier model families remains planned replication, not a completed cross-vendor result.

10.1 Task families

Retrieval retry probes (PTS, SMI); batch retrieval and scaling (BDTF); quantitative consistency (QI); structured visual interpretation (VMC); third-party UI/tool instructions (STK); phantom placeholders (PPG); artifact finalization and delivery (AFF, PPG, EBC); editorial boundary contamination (EBC); audience/context calibration (ACM).

10.2 Model coverage

Designed for parallel administration across frontier model families: Claude (Sonnet, Opus), GPT, Gemini, and any additional frontier model the investigator can access. Standardized prompts, retry prompts, tool access, and decoding settings are held constant.

10.3 Metrics

First-pass success; retry success; recoverability gap; artifact-readiness rate; numeric consistency rate; placeholder leakage rate; boundary-contamination rate; verification compliance rate; claim-source support rate; human intervention cost.

10.4 Procedure

Construct balanced item sets, administer matched prompts and retry prompts under standardized settings, hold tool access and decoding settings constant, administer retrieval items at batch sizes $N = 1, 5, 15, 30$, double-code annotations with Cohen's kappa or Krippendorff's alpha, and report Wilson confidence intervals, exact tests, and power statements.

10.5 Sample size and power

For a binary failure rate near 20% with desired margin of error ± 5 percentage points at 95% confidence, roughly 250 items per condition are required; for ± 3 points, roughly 700; for ± 2 points, roughly 1500. Detection of a 10 percentage-point across-model difference at 80% power requires approximately 200 items per arm.

11 Companion Repository and Reproducibility Package

The companion repository is maintained under the author’s GitHub account in the Research-Papers folder: <https://github.com/vjgits>. The repository folder for this paper is intended to serve as the archival location for the paper PDF, two-page public summary, experiment task pack, runner scripts, scorer scripts, raw outputs, scored CSV, and a concise results summary. Repository contents should be versioned with the paper version and model versions used in each run.

For the intra-vendor pilot, the minimal reproducibility bundle consists of: (i) `tasks/evaluation_tasks.jsonl`; (ii) `scripts/run_eval.py`; (iii) `scripts/score_results.py`; (iv) `results/raw_outputs.jsonl`; (v) `results/scored_results.csv`; (vi) a results summary; and (vii) a README listing model names, date of run, decoding settings, and API surfaces. Cross-vendor replication should append model outputs rather than overwrite prior outputs.

12 Mitigation Implications

The taxonomy suggests practical mitigations that do not require a fundamentally new model capability. Many of the modes arise because the system has not made verification, escalation, artifact readiness, and state preservation explicit. A model that can solve a task after a user asks it to retry can often be improved by making that retry policy internal, visible, and measurable.

For retrieval failures, the core mitigation is an explicit escalation policy: when the first search returns thin, the system should try query rewriting, site restriction, identifier search, and source triangulation before declaring failure. For batch tasks, the mitigation is batch-aware effort budgeting: the model should expose per-item verification status rather than producing uniform text that implies uniform depth.

For reasoning and artifact tasks, the highest-leverage mitigations are pre-delivery gates. Numeric responses should receive arithmetic self-checks; citation-heavy responses should receive claim-source entailment checks; file outputs should receive readiness checks for opening, formatting, links, placeholders, and boundary contamination. These are cheap compared with the downstream cost of a user discovering the issue after relying on the artifact.

For multimodal and interruption-related failures, the mitigations are procedural. Structured visual inputs should trigger a pause-and-confirm loop before downstream interpretation. Long-running tasks should maintain a progress ledger so that mid-response truncation, usage caps, or mobile streaming interruptions do not erase task state. Finally, evaluation pipelines should include counterfactual

replay so that first-pass success and retry-after-prompt success are reported separately.

13 Limitations and Threats to Validity

This paper is deliberately framed as a case-study-driven working paper with pilot experiments, not as a population-level model ranking. The naturalistic evidence base comes from a single user and is therefore vulnerable to selection effects. The same user who encountered the failures also helped identify and frame them, which introduces single-coder bias. Future versions should include independent double-coding and inter-annotator reliability.

The taxonomy is inductive and may be incomplete. It is also possible that some modes will collapse empirically once larger datasets are collected. PTS and SMI may co-occur frequently; AFF and EBC may form a broader artifact-readiness class; interruption-related modes may differ more by product layer than by model behavior. The current taxonomy should be understood as a measurement scaffold, not a final ontology.

Model attribution in conversation exports remains a threat to validity. The available export formats do not always include per-message model identifiers, making retrospective model comparison difficult. The paper therefore avoids strong claims from the naturalistic case study and relies on controlled pilot runs only where model identity is known. Even there, the intra-vendor pilot is small: two items per category and ten items per model. Its purpose is to demonstrate feasibility, not to estimate stable model rates.

The deterministic scoring rubric introduces its own measurement artifact. The `RETRY_PROXY` result shows that keyword-based scoring can misclassify semantically valid retrieval strategies when the model uses different vocabulary from the rubric. This limitation is not incidental; it is one of the key lessons of the pilot. Recoverable behavioral failures often involve policy quality rather than exact token presence, so scaled evaluation should combine deterministic checks with human annotation or semantic judging.

Finally, selective observability matters. These failures are easier to detect when users are expert enough to challenge the model. Less expert users may accept the first-pass failure as a true capability boundary and never trigger the recovery. That means the observed cases may understate the cost to ordinary users while overrepresenting workflows where the user is sufficiently engaged to notice.

14 Future Work

The immediate next step is to run the full cross-vendor protocol across Claude, GPT, Gemini, and additional frontier model families. The task pack should expand from ten pilot items to hundreds of items per family, including balanced retrieval tasks, artifact-production tasks, quantitative consistency tasks, multimodal extraction tasks, and public-communication boundary tasks.

Future work should also expand beyond a single-user corpus. A multi-user study with double-coded observations would allow estimation of mode incidence, co-occurrence, and user-visible cost. Longitudinal tracking across model-version releases would help distinguish model-side improvements from product-side mitigations such as better memory, better artifact audits, or better resume behavior.

A second line of work is infrastructure: open-source artifact-evaluation checks for downloadability, formatting, placeholder absence, link freshness, citation entailment, and boundary contamination. These checks would make professional-document and code-artifact evaluation more concrete than ordinary text-preference scoring.

A third line of work is product feedback design. Providers could include per-message model identifiers in exports, capture retry traces in feedback flows, and expose recoverability metrics alongside standard accuracy and preference metrics. Such instrumentation would make RBFs visible at scale rather than relying on users to notice and report them manually.

15 Conclusion

Recoverable behavioral failures are cases where a model appears capable of completing a task but fails to exercise that capability on the first pass. They are not captured by standard terminal-output evaluation, because they become visible only when a retry, correction, or artifact audit reveals that the model could have produced the correct output without new substantive information. This paper contributes a definition, a layered sixteen-mode taxonomy, a quantitative retry-probe pilot with exploratory confidence intervals and an exact small-sample association test, abstracted cross-platform observations, an intra-vendor pilot across three Anthropic models, and a cross-vendor evaluation protocol with supporting analysis scripts. AI evaluation should measure not only capability but also persistence, verification, escalation, artifact readiness, and boundary control.

References

- Bai, Yuntao, Andy Jones, Kamal Ndousse, Amanda Askell, Anna Chen, Nova DasSarma, Dawn Drain, Stanislav Fort, Deep Ganguli, Tom Henighan, Nicholas Joseph, Saurav Kadavath, Jackson Kernion, Tom Conerly, and others. 2022. Training a Helpful and Harmless Assistant with Reinforcement Learning from Human Feedback. arXiv:2204.05862.
- Chen, Wenhui, Xinyi Wang, and William Yang Wang. 2021. A Dataset for Answering Time-Sensitive Questions. arXiv:2108.06314.
- Christiano, Paul F., Jan Leike, Tom B. Brown, Miljan Martic, Shane Legg, and Dario Amodei. 2017. Deep Reinforcement Learning from Human Preferences. NeurIPS.
- Hendrycks, Dan, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2021. Measuring Massive Multitask Language Understanding. ICLR.
- Ji, Ziwei, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. 2023. Survey of Hallucination in Natural Language Generation. ACM Computing Surveys.
- Jimenez, Carlos E., John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? ICLR.
- Liang, Percy, Rishi Bommasani, Tony Lee, Dimitris Tsipras, Dilara Soylu, Michihiro Yasunaga, Yian Zhang, Deepak Narayanan, Yuhuai Wu, Ananya Kumar, Benjamin Newman, and others. 2022. Holistic Evaluation of Language Models. arXiv:2211.09110.
- Lin, Stephanie, Jacob Hilton, and Owain Evans. 2022. TruthfulQA: Measuring How Models Mimic Human Falsehoods. ACL.
- Liska, Adam, Tomas Kocisky, Elena Gribovskaya, Tomáš Terzi, Eren Sezener, Dhruv Agrawal, and Phil Blunsom. 2022. StreamingQA: A Benchmark for Adaptation to New Knowledge over Time. ICML.
- Liu, Xiao, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Yuxian Gu, Hangliang Ding, Kaiwen Men, and others. 2023. AgentBench: Evaluating LLMs as Agents. arXiv:2308.03688.
- Mialon, Grégoire, Clémentine Fourrier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. 2023. GAIA: A Benchmark for General AI Assistants. arXiv:2311.12983.
- Nakano, Reiichiro, Jacob Hilton, Suchir Balaji, Jeff Wu, Long Ouyang, Christina Kim, Christopher Hesse, Shantanu Jain, Vineet Kosaraju, William Saunders, and others. 2021. WebGPT: Browser-Assisted Question-Answering with Human Feedback. arXiv:2112.09332.
- Park, Peter S., Simon Goldstein, Aidan O’Gara, Michael Chen, and Dan Hendrycks. 2024. AI Deception: A Survey of Examples, Risks, and Potential Solutions. Patterns 5(5).
- Patil, Shishir G., Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. 2023. Gorilla: Large Language Model Connected with Massive APIs. arXiv:2305.15334.
- Qin, Yujia, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, and others. 2023. ToolLLM: Facilitating Large Language Models to Master 16,464 Real-World APIs. arXiv:2307.16789.
- Rafailov, Rafael, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. 2023. Direct Preference Optimization: Your Language Model is Secretly a Reward Model. NeurIPS.
- Schick, Timo, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language Models Can Teach

Themselves to Use Tools. NeurIPS.

Sharma, Mudit, Meg Tong, Tomasz Korbak, David Duvenaud, Amanda Aspell, Samuel R. Bowman, Esin Durmus, Zachary Hatfield-Dodds, Scott R. Johnston, and Ethan Perez. 2023. Towards Understanding Sycophancy in Language Models. arXiv:2310.13548.

Shinn, Noah, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. NeurIPS Workshop / arXiv:2303.11366.

Srivastava, Aarohi, Abhinav Rastogi, Abhishek Rao, Abu Awal Md Shoeb, Abubakar Abid, Adam Fisch, Adam R. Brown, Adam Santoro, Aditya Gupta, Adrià Garriga-Alonso, and others. 2023. Beyond the Imitation Game: Quantifying and Extrapolating the Capabilities of Language Models. TMLR.

Wang, Guanzhi, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2023. Voyager: An Open-Ended Embodied Agent with Large Language Models. arXiv:2305.16291.

Wei, Jason, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V. Le, and Denny Zhou. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. NeurIPS.

Yao, Shunyu, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. ICLR.

Yin, Robert K. 2018. Case Study Research and Applications: Design and Methods. 6th ed. SAGE.

Zhou, Shuyan, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. 2024. WebArena: A Realistic Web Environment for Building Autonomous Agents. ICLR.

Appendix A. Observation Bank Coding Schema

Each candidate event in the observation bank is coded with: `observation_id`, `task_family`, `original_prompt`, `first_pass_output`, `failure_criterion`, `retry_prompt`, `new_information_introduced`, `post_retry_output`, `mode_code`, `trigger`, `recovery_type`, `user_visible_cost`, `evidence_tier`. Conservative coding rule: if recovery requires new substantive information, the event is excluded.

Appendix B. Experiment Pack and Task Schema

The companion pack contains ten items across five categories: QI (2), PPG (2), EBC (2), ACM (2), and RETRY_PROXY (2), plus runner and scorer scripts for OpenAI, Anthropic, and Google API surfaces. Per-item fields include `id`, `category`, `prompt`, and `checks`: forbidden terms, required terms, or category-specific numeric expectations. Scoring uses deterministic checks where possible; remaining items are double-coded by independent annotators.

Appendix C. Annotation Rubric

- Did the first-pass output satisfy the task’s acceptance criteria? If yes, label first-pass success.
- If failed, did the retry succeed?
- Did the retry prompt introduce new substantive information? If yes, label ordinary interactive correction.
- If retry succeeded without new information, label recoverable behavioral failure and assign a mode code.
- If retry also failed, label non-recoverable under available context.
- If unclear, label ambiguous.

Appendix D. Reproducibility Checklist

- Item set with stable IDs, prompts, retry prompts, expected outputs, and verification methods.
- Model and tool settings recorded.
- Raw outputs archived in append-only JSONL with timestamps.
- Scorer script with deterministic check criteria.
- Annotation guidelines and double-coded labels with reported kappa or alpha.
- Analysis notebook producing all tables, confidence intervals, and statistical tests.

- Data statement: collection method, privacy controls, redaction, and licenses.
- Versioning: pack version, model versions, and analysis-notebook commit hash.

Appendix E. Verification-Trigger Control Design

The retry-probe design separates first-pass success from success after a prompted retry. A key validity concern is that a second attempt may succeed merely because the model receives another sample, additional decoding budget, or an opportunity to rephrase. To isolate the behavioral trigger, the benchmark defines matched retry conditions that distinguish random second-pass recovery from directed verification and escalation.

For each item that fails on first pass, the evaluation routes the same item through post-failure conditions: a null retry, a generic retry, and a verification retry. The null retry asks the model to continue or answer again without explicitly requesting verification. The generic retry asks the model to try again. The verification retry asks the model to inspect whether it missed a requirement, search differently, verify calculations, or audit the artifact before answering. The primary quantity is directed recoverability lift: the difference between verification-retry success and null-retry success.

Condition	Prompt family	What it tests	Primary metric
First pass	Original user task	Baseline execution behavior	First-pass success
Null retry	Please continue. / Please answer again.	Second-sample effect, extra tokens, stochastic recovery	Null-retry success
Generic retry	Try again.	Broad retry without explicit audit cue	Generic-retry success
Verification retry	Check whether you missed anything; verify, escalate, or audit before answering.	Directed verification and escalation trigger	Verification-retry success
Self-audit first pass	Before finalizing, verify every requirement.	Whether front-loaded audit reduces RBFs	Audit-first success

The design yields four separable quantities: first-pass success p_1 ; null-retry success p_n ; generic-retry success p_g ; verification-retry success p_v ; and directed recoverability lift $L = p_v - p_n$. If L is near zero, apparent recoverability may mostly reflect stochastic second-pass sampling. If L is positive, recoverability depends on an explicit behavioral trigger. This distinction separates capability revealed by another sample from capability revealed by a specific verification or escalation instruction.

E.1 Statistical plan for the control design

Each item is assigned to retry conditions using a paired or blocked design. For retrieval items, the same canonical item can be evaluated across models using independent sessions and randomized condition order to reduce memory and contamination effects. The primary analysis reports paired success differences, Wilson confidence intervals for each condition, and McNemar or exact paired tests where item-level pairing is preserved. For cross-model comparisons, mixed-effects logistic regression can model success as a function of model, mode family, retry condition, batch size, and item difficulty, with random intercepts for item.

Recommended reporting includes p_1 , p_n , p_g , p_v , $L = p_v - p_n$, and RBF rate under each retry policy. Verification-retry success is interpreted alongside null-retry baselines rather than as pure capability in isolation.

Appendix F. Mechanistic Hypotheses for Recoverable Behavioral Failures

The RBF taxonomy is observational, but it motivates mechanistic hypotheses that can be tested empirically without requiring privileged access to model internals. The purpose of this layer is not to assert a single cause for all RBFs, but to specify distinct predictions that can be evaluated through controlled prompts, repeated sampling, query logs, artifact audits, and, where available, token-probability

traces.

Hypothesis	Likely modes	Prediction	Possible test
Sampling drift	QI, VMC, some PPG	Failures disappear across repeated null retries without directed cues.	Repeated-sample baseline at fixed temperature and prompt.
Escalation-threshold policy	PTS, SMI, STK	Directed verification retry outperforms null retry; model had strategy available but did not trigger it.	Null vs verification retry lift; query-log analysis.
Batch effort allocation	BDTF, PTS, SVD	Failure rate rises with batch size even when single-item accuracy remains high.	Batch-scaling curve N=1,5,15,30.
Context allocation / salience loss	VMC, SVD, ACM	Late, buried, or visually complex requirements are missed more often than salient ones.	Position-controlled prompts; structured visual extraction probes.
Artifact-boundary weakness	AFF, PPG, EBC	Textual answer quality is high while deliverable-level readiness fails.	Artifact linting and boundary-audit checks.
Product/system interruption	MRT, MSI, UCT	Failures cluster near token, stream, or rate-limit boundaries.	Telemetry on stream termination and resume success.

These hypotheses are execution-policy explanations rather than psychological claims about the model. For frontier API models, token-probability analysis may be limited by vendor support for logprobs. When logprobs are available, analysis can compare probability mass around failure tokens (for example, unresolved, unavailable, cannot determine) versus escalation tokens (site search, exact match, verify, retry) across first-pass and verification-retry conditions. When logprobs are unavailable, repeated-sample baselines, directed-retry controls, batch-scaling curves, and artifact-readiness checks still provide external evidence about execution policy.

F.1 Mode-to-hypothesis mapping

PTS and SMI primarily test escalation-threshold policy. BDTF tests batch effort allocation. QI tests self-audit failure and, under repeated null retries, possible sampling drift. VMC tests context allocation and structured visual salience. AFF, PPG, and EBC test artifact-boundary weakness. MRT, MSI, and UCT test system interruption and resume-state reliability. SVD tests source-verification discipline and context allocation across long research workflows. ACM tests audience-frame persistence and boundary adherence.

Appendix G. RBF-Eval Benchmark Suite Specification

RBF-Eval is specified as a reusable benchmark scaffold for measuring recoverable behavioral failures in LLM workflows. It uses a stable JSONL schema, a runner abstraction, deterministic and human-judged scoring hooks, and contribution guidelines. The goal is to make the taxonomy executable: researchers should be able to add items, run matched first-pass and retry conditions, and compare models using common metrics.

```
rbf-eval/  
  README.md  
  pyproject.toml  
  rbf_eval/  
    __init__.py  
    schemas.py  
    runner.py  
    scorer.py  
    metrics.py  
    analysis.py  
    prompts.py  
  tasks/  
    rbf_eval_v0_1.jsonl
```

```

prompts/
  first_pass.txt
  null_retry.txt
  generic_retry.txt
  verification_retry.txt
results/
  raw_outputs.example.jsonl
  scored_results.example.csv
  results_summary_template.md
notebooks/
  analyze_results.ipynb
docs/
  taxonomy.md
  annotation_rubric.md
  contribution_guide.md
  schema_reference.md
  runbook.md
paper/
  Observed_Recoverable_Behavioral_Failures_LLM_Workflows_v1_14.pdf

```

The benchmark supports three evaluation flows: first-pass only, first-pass plus null retry, and first-pass plus verification retry. The scorer emits item-level outcomes and aggregate metrics by model, task family, failure mode, and retry condition. This design lets RBF-Eval distinguish stochastic recovery from directed verification lift while preserving compatibility with deterministic and human-judged tasks.

G.1 Standard task schema

```

{
  "id": "pts_opaque_001",
  "mode": "PTS",
  "task_family": "retrieval_retry",
  "prompt": "...",
  "expected_behavior": "resolve item or report verified failure after escalation",
  "first_pass_acceptance": {...},
  "retry_conditions": {
    "null_retry": "Please continue.",
    "generic_retry": "Try again.",
    "verification_retry": "Check whether you missed anything; try a different verification or retrieval strategy."
  },
  "checks": {
    "required_terms": [],
    "forbidden_terms": [],
    "numeric_expectations": [],
    "artifact_checks": [],
    "human_judged": true
  },
  "metadata": {
    "source_type": "synthetic_or_public",
    "privacy": "non_sensitive",
    "difficulty": "medium",
    "version": "0.1"
  }
}

```

This schema separates mode, task family, acceptance criteria, retry conditions, and metadata. The separation is necessary because the same failure mode can appear in multiple task families, and the same task can be evaluated under multiple retry conditions.

G.2 Companion artifact contents

RBF-Eval is designed to be released as a companion artifact alongside the paper. The release package should include the paper PDF, benchmark package, task file, retry-prompt templates, runner and scorer scripts, documentation for the taxonomy and annotation rubric, example raw-output and scored-result files, and a results-summary template. The small Anthropic intra-vendor pilot reported in this paper is included as an illustrative pilot result; full cross-vendor scaling remains planned replication work.